CPU AND MEMORY STRESS TESTING TOOL

Section 6 , Team 1

Team members:

2520030317 - Navaneeth

2520030317 - Himesh

Faculty: Dr. Kotte Anusha

PROBLEM STATEMENT

•Computers often slow down under heavy CPU load. This affects performance and responsiveness.

•High memory usage can cause paging and swapping. This leads to delays and reduced efficiency.

•The operating system must manage resources effectively. Scheduling and allocation are critical tasks.

•Stress testing helps observe system behavior. It creates controlled extreme conditions.

•The project simulates CPU and memory stress. It measures how the OS responds.

•The goal is to analyze performance bottlenecks. This provides insights into OS resilience.

OBJECTIVES

•Build a functional prototype that runs entirely in terminal. This avoids UI complexity and focuses on backend logic.

•Implement CPU stress testing by generating continuous mathematical workloads. This keeps processor cores busy.

•Implement Memory stress testing by allocating and manipulating RAM blocks. This simulates heavy memory usage.

•Provide a combined test that stresses both CPU and memory simultaneously. This shows dual resource pressure.

•Monitor real-time CPU and RAM usage during tests. This uses system information from /proc.

•Display results clearly in terminal after each test. This ensures transparency and easy analysis.

PROJECT FLOW

•The CPU Tester module creates workloads that keep cores busy. It uses multiple threads for concurrency.

•The Memory Tester module allocates and manipulates RAM. It simulates realistic memory stress conditions.

•The Monitor module reads system statistics from /proc/stat and /proc/meminfo. It tracks CPU and memory usage.

•The Results module displays usage statistics and logs test completion. It confirms resource release.

•Execution flow: Menu → Select test → Run workload → Monitor → Results. This ensures structured operation.

•The design is modular and extensible. Future features can be added without major changes.

IMPLEMENTATION HIGHLIGHTS

- The tool is implemented in C language on Ubuntu. This ensures low-level OS interaction.
- Threads are used to simulate CPU stress. They demonstrate concurrency and scheduling.
- Processes are managed with fork and signals. This shows OS process control.
- Memory allocation is handled with malloc and mmap. This stresses RAM usage and paging.
- System calls are used for monitoring and logging. They provide direct kernel interaction.
- Synchronization is achieved with mutex and semaphores. This prevents race conditions and ensures safe execution.

OUTCOMES & LEARNINGS

•The tool demonstrates OS scheduling under CPU load. Threads compete for processor time.

•It shows memory allocation and paging behavior. RAM usage and swapping are observed.

•It highlights synchronization and concurrency concepts. Mutex and semaphores ensure safe execution.

•It validates signal handling and resource cleanup. Tests stop gracefully on interrupts.

•It provides a practical stress testing utility. This can be reused for analysis and benchmarking.

•It strengthens understanding of core OS principles. This is valuable for hackathon learning and evaluation.

FUTURE SCOPE

- Add dynamic CLI arguments for threads, memory size, and duration. This increases flexibility.
- Extend tool to include disk I/O stress testing. This adds another resource dimension.
- Integrate live monitoring dashboards with graphical plots. This improves visualization.
- Package as a benchmarking utility for performance testing. It can compare system efficiency.
- Explore network stress testing for distributed systems. This expands scope to communication workloads.
- Enhance portability across different operating systems. This increases usability and reach.

CONCLUSION

•Motto: *“Demonstrating OS resilience under extreme workloads.”* This summarizes the project purpose.

•The tool integrates CPU stress, memory stress, monitoring, and results. It is comprehensive.

•It is efficient, modular, and hackathon-ready. Judges can see practical relevance.

•It demonstrates Operating System principles clearly. Scheduling, memory, and synchronization are covered.

•It provides valuable insights into resource management. Performance bottlenecks are visible.

•It is a practical utility for testing and learning. It bridges theory and practice effectively.


Thank
you!